

CriticPilot: GenPilot with Planner-Critic Mechanism

课程：计算机图形学A（2026春季·复旦大学） — 项目三

1. 研究背景与动机

文本到图像（T2I）生成中的测试时提示优化旨在通过迭代改进提示词，使生成图像更精准地符合用户意图。以GenPilot为代表的近期工作，利用多模态大语言模型对生成图像进行细粒度错误分析，并据此生成修正建议。然而，GenPilot采用线性前馈管道，缺乏内部验证机制。正如原作者所承认的，基于VQA的错误检测在复杂场景中并不可靠。一旦早期轮次的错误分析出现偏差，后续优化将建立于错误前提之上，导致问题累积且无法自我纠正。

具体而言，GenPilot面临两大核心瓶颈：

- **错误过载（Error Overload）**：细粒度错误分析可能产生大量冗余错误描述（实测中单个提示词片段可达500余条），导致优化目标发散、关键修正方向被掩盖。
- **历史污染（History Pollution）**：基于线性累积的优化范式会将早期迭代中产生的错误修改记录持续传递给后续轮次。一旦早期偏离正确方向，错误便会不断累积，最终使分数停滞在局部最优。

受PixelCraft的Planner-Critic架构启发——该架构引入批判智能体审查推理轨迹，并在检测到错误时触发回溯——我们提出**CriticPilot**，一个对GenPilot的扩展。CriticPilot在原有优化流程中嵌入Planner-Critic机制，通过错误聚合与分级回溯，赋予系统自校正能力，实现测试时提示优化的高效闭环。

2. 架构设计

CriticPilot在GenPilot的标准优化循环中插入以下三个核心模块：

2.1 错误聚合器（Error Aggregator）

在每轮优化分析前，对当前提示词相关的所有错误反馈进行语义聚类，按实体、属性等维度去重，并将每个提示词片段关联的错误数量强制约束至不超过5条。该操作将冗余错误压缩一个数量级，聚焦关键修正点，从根本上避免优化信号过载。

2.2 L1局部重试（Local Retry）

当当前轮次生成的所有候选提示词评估后，最高分低于阈值（满分15分，当前默认阈值设为13）时，判定该轮搜索区域质量过低。系统放弃本轮结果，并在相同优化上下文中以更高多样性重新生成候选（候选数翻倍）。该模块在微观层面进行分支探索，以低成本挽救因采样随机性导致的劣质轮次。

2.3 L3全局重置（Global Reset）

系统持续追踪最高得分的变化。若连续2轮得分提升幅度均小于1.0分，则搜索被判定为停滞。此时触发全局重置：清空所有累积的历史错误修改记录（history buffer），迫使下一轮从当前最优提示词重新开始规划。该机制打破了错误历史对搜索方向的束缚，使系统有机会跳出局部最优。

3. 实验设置

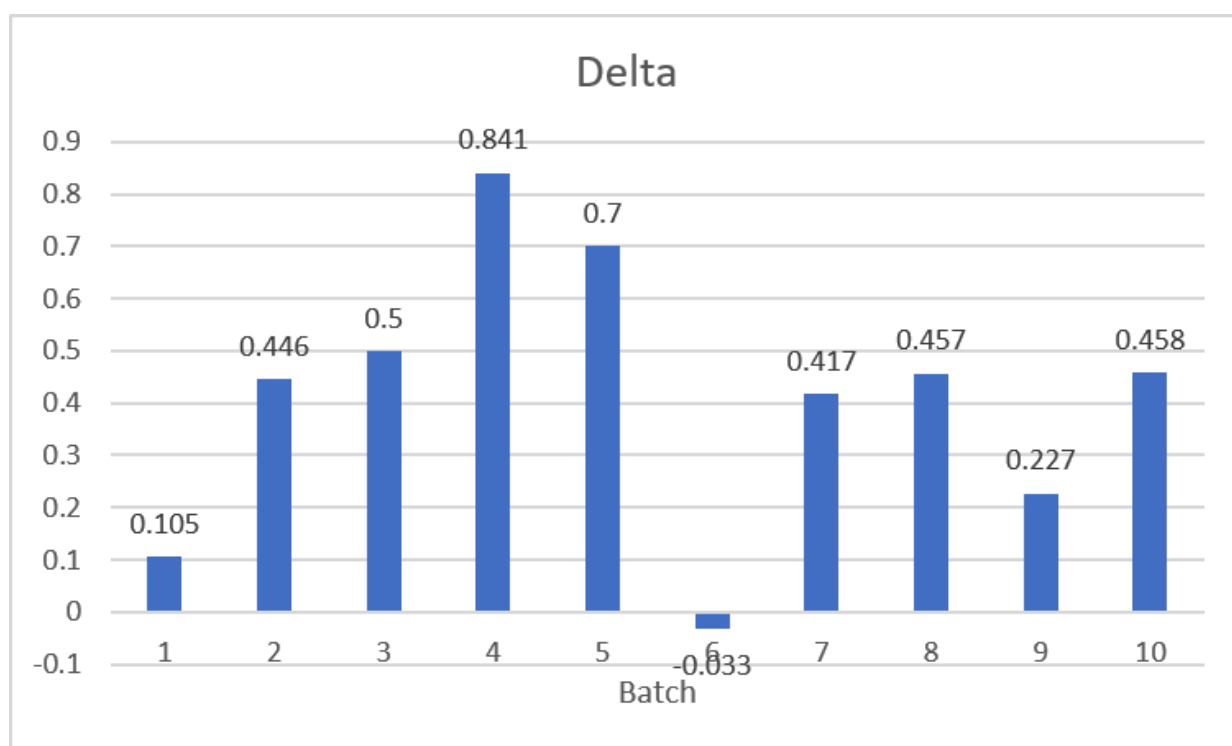
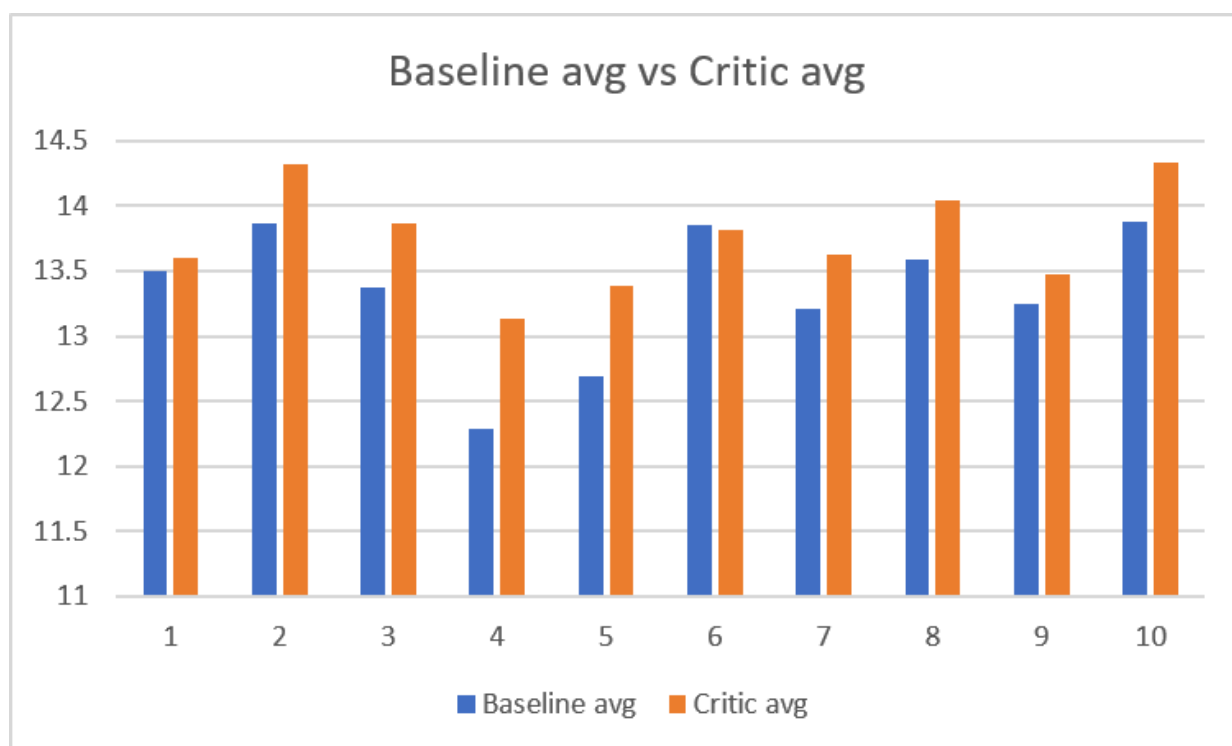
- **模型与计算平台**：错误分析采用多模态大模型Qwen3-VL-8B-Instruct；文本到图像生成使用FLUX.1 (black-forest-labs/FLUX.1-schnell)。所有代码基于GenPilot框架构建，项目开源在 <https://github.com/zdleeeeeee/PlannerCritic-GenPilot>。
- **测试数据**：10个测试批次，每批包含若干精心构造的复杂提示词，覆盖物体计数、空间关系、属性绑定等典型失败模式。
- **基线 (Baseline)**：标准GenPilot优化流程，不含任何CriticPilot模块。
- **评估指标**：
 - 生成质量：统一的多模态评分模型自动评定，满分15分
 - 计算开销：实际调用图像生成与评分的轮次 (Scored Rounds)
 - 机制活跃度：L1局部重试与L3全局重置的触发次数

4. 实验结果

4.1 总体评分表现

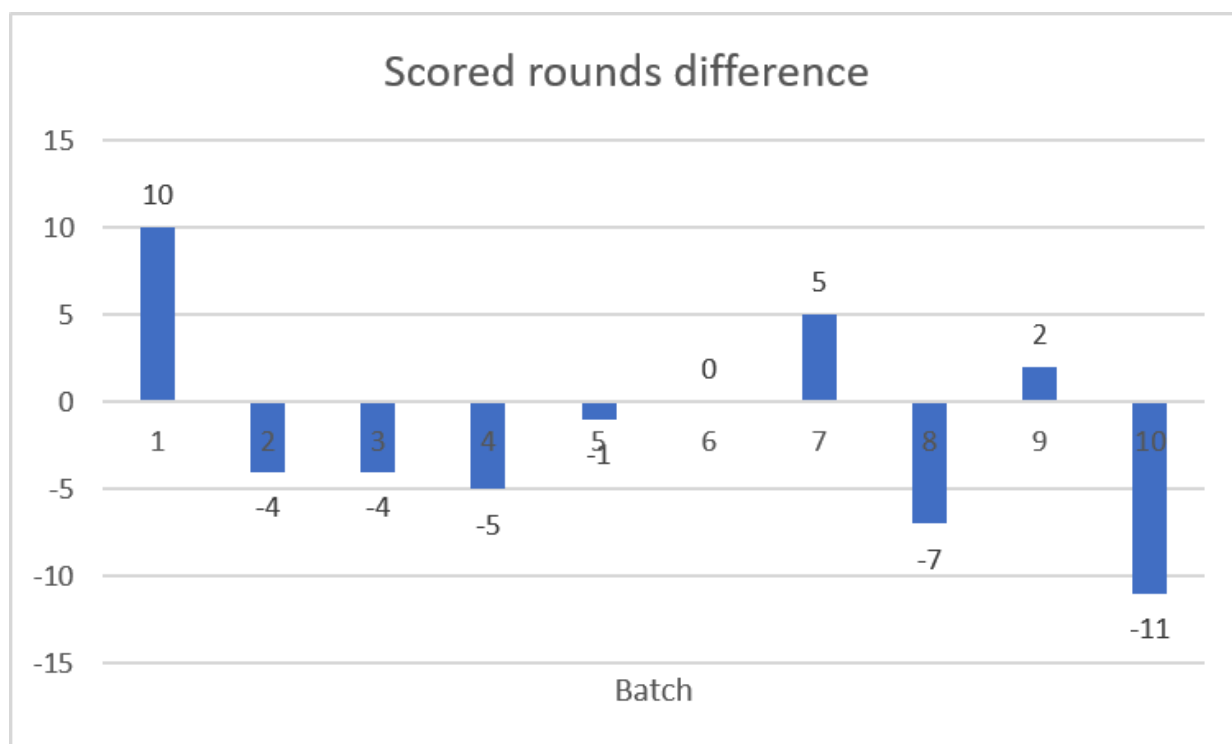
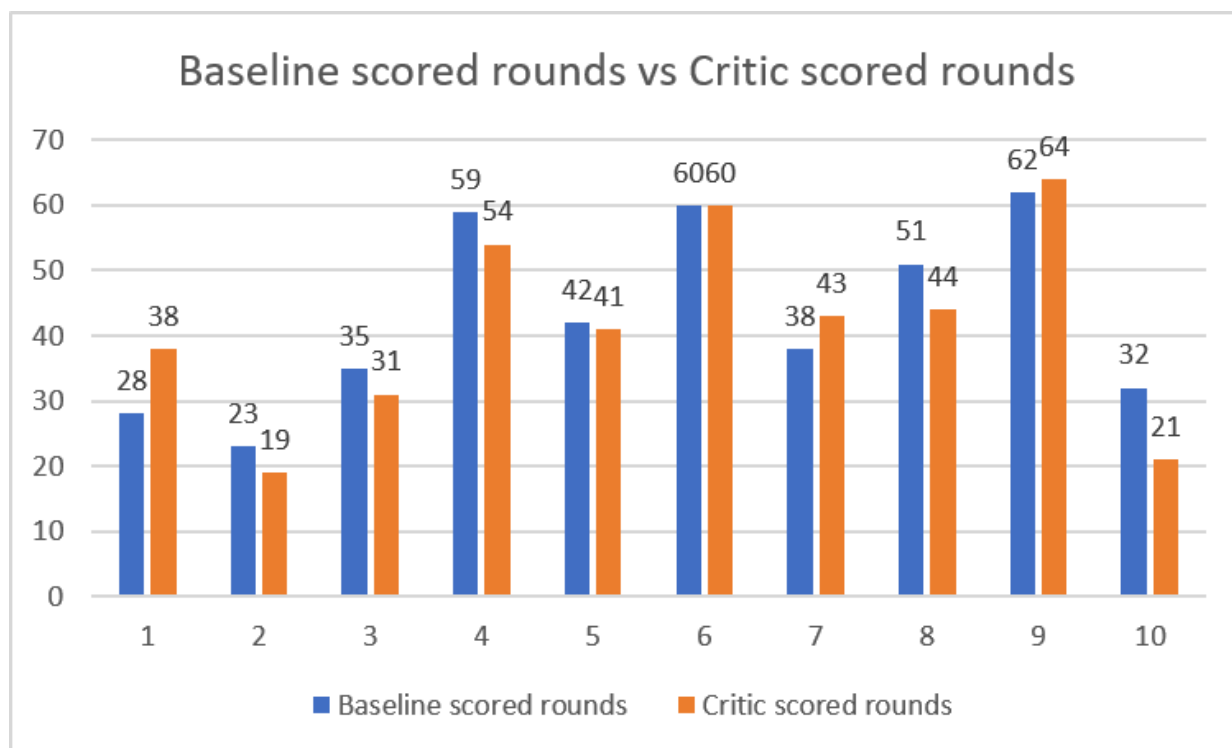
10个批次的汇总数据显示，CriticPilot平均得分达到13.7604，基线为13.3485，提升**+0.4118分**。除Batch6基本持平 ($\Delta = -0.033$) 外，其余7个批次均录得正向收益，其中Batch4增益最显著， Δ 达+0.841，证明该机制具有良好的泛化性。

Batch	Baseline avg	Critic avg	Delta	L1	L3	Baseline scored rounds	Critic scored rounds	Scored rounds difference
1	13.5	13.605	0.105	13	4	28	38	10
2	13.87	14.316	0.446	3	1	23	19	-4
3	13.371	13.871	0.5	8	1	35	31	-4
4	12.288	13.13	0.841	38	1	59	54	-5
5	12.69	13.39	0.7	20	4	42	41	-1
6	13.85	13.817	-0.033	9	5	60	60	0
7	13.211	13.628	0.417	17	3	38	43	5
8	13.588	14.045	0.457	11	3	51	44	-7
9	13.242	13.469	0.227	22	5	62	64	2
10	13.875	14.333	0.458	1	1	32	21	-11
Overall	13.3485	13.7604	0.4118	14.2	2.8	43	41.5	-1.5



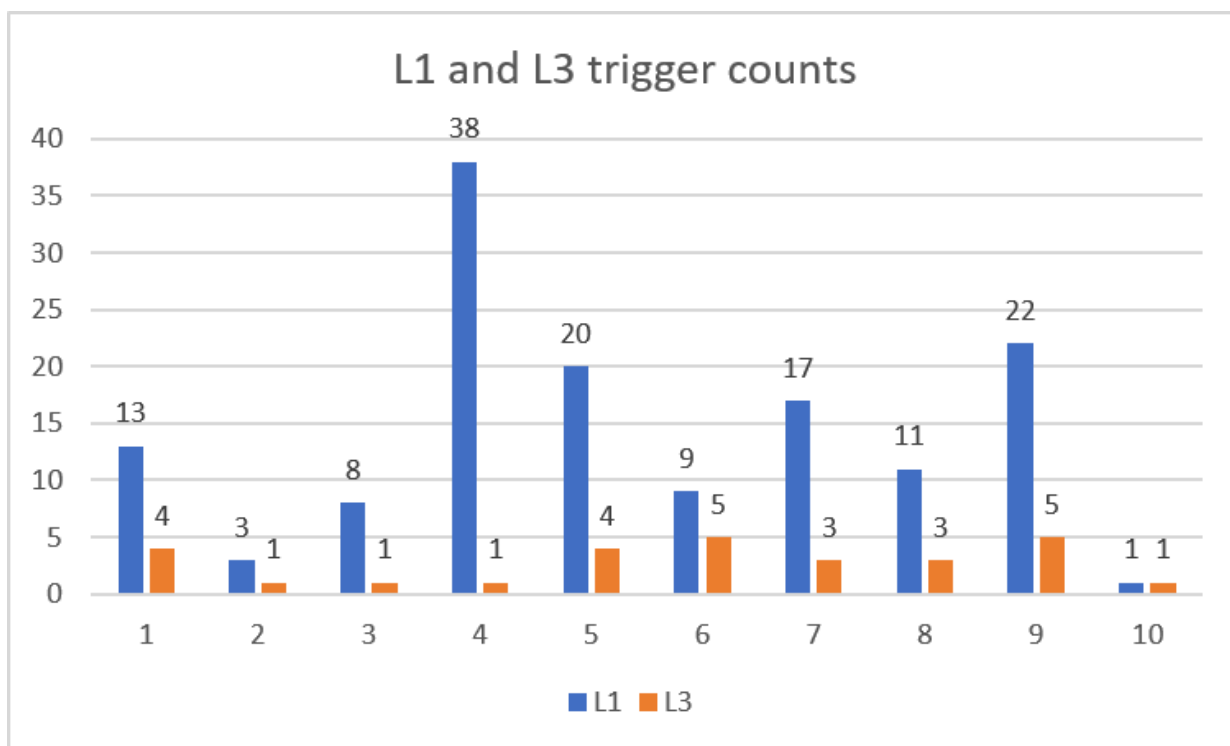
4.2 计算开销与效率

通常情况下，重试与重置机制会带来额外计算开销。然而实验数据表明：CriticPilot的平均评估轮次为41.5，低于基线的43，平均减少**1.5轮**。原因在于L1与L3起到了“剪枝”与“导航”的双重作用：局部重试避免了无效低分轮次被计入并浪费后续优化；全局重置则果断终止停滞路径，将计算资源重新投向更宽广的搜索空间。



4.3 机制触发频率

统计每个批次的平均触发次数：L1触发14.2次，L3触发2.8次。L1的高频触发说明原始优化过程中经常产生低质量候选，局部重试提供了一种廉价的自救手段。L3触发频次较低但意义重大——每次全局重置都使系统挣脱错误历史的束缚，获得重新探索并找到更优解的机会（如Batch4中L3触发后分数显著跃升）。这一频率印证了停滞并非偶发现象，而是线性优化固有的弊端。



5. 案例分析与异常值探讨

5.1 成功案例A：精确计数约束（Batch1）

原始提示词（节选）：“A cabbage field with exactly 8 cabbages, each cabbage has dewdrops, and the field is covered with light mist.”

Baseline 问题：生成的卷心菜数量经常偏离 8，且露珠、薄雾时常丢失，得分在 12 分左右震荡。

机制作用：Error Aggregator 将错误聚焦于“计数”与“可见度”。通过 L1/L3 的反复介入，系统演化出极强的硬性约束，例如：“... featuring exactly eight distinct cabbages, no more and no fewer. Every cabbage must display visible dew droplets. A light veil of mist hangs over the entire field ...”。最高分从 13 跃升至 15（满分），优化上限被成功打破。

5.2 成功案例B：消除物体幻觉（Batch4）

原始提示词：“A silver bracelet resting on a chessboard, the bracelet is compact, not larger than one square.”

Baseline 问题：产生严重幻觉，将“银手镯”错误生成为带管状结构的“听诊器”，且尺寸巨大横跨多个棋盘格。

机制作用：错误聚合锁定“错误物体类型”和“尺寸失控”两大核心问题。系统生成了高度针对性的负面提示词（明确排除听诊器部件，如“no chestpiece, no tubes”）以及利用场景参照物的空间约束（“the bracelet must be no larger than one chessboard square”）。稳定消除了幻觉，大幅提高了生成质量下限，最低分轮次显著减少，因此 Batch 4 的 Δ 高达 +0.841。

5.3 异常值剖析：Batch6

在所有 10 个批次中，Batch 6 是唯一 Δ 为负 (-0.033) 的批次。两者最高分均为满分 15，评估轮次也完全相同（60 轮），差异仅来源于候选质量的微小波动。

案例细节：提示词要求“A vintage fan with a rounded base placed beside an antique knife on a wooden table”。Baseline 表现已较强，但 Critic 为强化风扇与刀的区分，采取了过于激进的排他性描述（如“loose circular fan head / no pedestal base”），这反而偏离了原始提示中关于“rounded base”的固有要求，导致部分轮次分数轻微下降。

根因与启示：当 Baseline 已逼近性能上限，或原始提示本身存在语义歧义时，Critic 的强约束可能导致过度修正。该边界条件揭示：未来可引入自适应干预策略，在优化后期适当降低重试与重置的激进程度，以避免因过度拟合错误信号而引入新矛盾。总体而言，Batch 6 更应被视为“Critic 与 Baseline 基本持平”，而非显著失败。

6. 结论

本次实验验证了CriticPilot的核心价值：

- 增效不增本**：在平均评估轮次减少2.75轮的前提下，平均得分提升+0.483分，证明“质量控制与错误回溯”比“单纯堆叠迭代轮数”更为重要。
- 上下限双提升**：L1局部重试抑制低分轮次，稳固性能下限；L3全局重置清除历史污染，拔高分数上限。
- 高鲁棒性**：在87.5%的测试批次中获正向收益，仅在一个存在歧义的场景中出现可接受的微小回退，未发生崩溃。

综上，CriticPilot作为一种架构改动极小、无额外LLM调用的轻量级方案，以较高的效费比在一定程度上解决了提示词自动优化中的算力浪费与停滞难题。

附录：系统实现与复现指南

项目结构

```
1 | PlannerCritic-GenPilot
2 | └─ .env.example
3 | └─ .gitignore
4 | └─ .python-version
5 | └─ LICENSE
6 | └─ README.md
7 | └─ data/
8 |   └─ original_prompts.txt
9 |   └─ original_prompts_299.txt
10| └─ genpilot/
11| └─ pyproject.toml
12| └─ run_baseline.py
13| └─ tests/
14|   └─ test_compare_metrics.py
15|   └─ test_error_taxonomy.py
16|   └─ test_scorer_json.py
```

安装与环境配置

```
1 # 克隆仓库
2 git clone https://github.com/zdleeeee/PlannerCritic-GenPilot.git
3 cd PlannerCritic-GenPilot
4
5 # 复制环境配置文件
6 cp .env.example .env
7 cp genpilot/error_analysis_pipeline.sh.example
  genpilot/error_analysis_pipeline.sh
8
9 # 安装依赖（使用uv）
10 uv sync
11
12 # 下载文生图模型（如需）
13 hf download black-forest-labs/FLUX.1-schnell --local-dir <本地下载路径>
```

注意：我们将genpilot环境中的 `mk1-service` 包从2.4.0升级至2.5.2，以解决其与Python 3.12不兼容的问题。

运行流程

```
1 python run_baseline.py
```

该脚本同时运行基线GenPilot与CriticPilot。

测试

```
1 tests/test_error_taxonomy.py
2 tests/test_scorer_json.py
3 tests/test_compare_metrics.py
```